CSC-225: Extend the LoginMonitor

Individual coding assignment | Advanced Python Programming

Assigned	September 29, 2026
Due	October 10, 2026 at 11:59 PM
Work type	Individual — your own code, your own submission
Starter code	Posted separately on the course's Canvas page

Questions? Reach out to casey.hannan@bhcc.edu

What This Is

For the last couple of class sessions, we've built **LoginMonitor** together as a class — a tool that reads login-attempt data and reports on failed logins, suspicious IPs, and locked-out accounts. This assignment picks up exactly where we left off. You'll add four new capabilities to the same class, using the same log data we've already worked with.

Nothing here requires anything we haven't covered. The building blocks are all things you've already written in class: loops over `self.parsed_logs`, `if` filters on `record["status"]`, the `.get(key, 0)` tally pattern, `sorted()` with a `lambda` key, and default arguments. The one genuinely new tool is the **set** (`{...}`) — the starter file explains it where you need it.

What You're Starting From

Your instructor has posted **assignment_starter_login_monitor.py** separately on the course's **Canvas** page. It already contains a fully working `LoginMonitor` class with:

- `parse_line() / parse_logs()`
- `failed_attempts_by_user()`
- `most_suspicious_ip()`
- `locked_accounts(threshold=3)`
- `summary_report()`

Run it as-is first and confirm you get the same output we saw in class. If you don't, **stop and fix that before writing anything new** — you'll be building on top of this code, and it needs to be correct first.

What You Need to Build

Four new methods, each specified as a docstring (with a `# TODO` marker) inside the starter file. Full specs are in the code; here's the summary:

1. unique_ips(self, username)

Return the **set** of distinct IP addresses a given username has ever connected from. Returns a set, not a list — order doesn't matter here, and a set automatically won't contain duplicates, which is exactly what we want.

2. multi_ip_users(self, min_ips=2)

Return a sorted list of usernames who have connected from at least `min_ips` different IP addresses — a real signal security teams watch for (could mean a shared password, could mean a compromised account). Build this using `unique_ips()` — don't re-loop through raw records from scratch.

3. busiest_hour(self)

Every timestamp looks like `"2026-09-10 08:15:02"`. Figure out which **hour** (`"08"`, `"09"`, etc.) has the most total login attempts — successes and failures both count here, this isn't about failures. Return a tuple: `(hour, count)`.

4. generate_alerts(self)

Pull the other methods together into a list of human-readable alert strings — this is your "so what" method, the one a security analyst would actually want to read. At minimum, it should report locked accounts, the most suspicious IP, and any multi-IP users. Formatting is up to you, but each entry should be a complete, readable sentence.

Testing Your Work

For **each** method, add at least one `print()` statement below the class that calls it and shows the output — the same "STOP & TEST" pattern we've used in every session. We are not looking for a formal test framework here, just clear evidence, right in the file, that you ran your code and checked the result.

Submission

1. Submit your work as either a **ZIP file** of your project folder **or** a link to a **GitHub repository** — your choice.
2. Include a short **README.md** in your submission with: what the project does (2-3 sentences), how to run it, and **one paragraph explaining your busiest_hour() logic in your own words**. This isn't a trick — it's a chance to show you understand what you wrote, which matters more than the code itself (see the AI Use note below).
3. Submit the ZIP file or the GitHub repo link on **Moodle** by the due date.

A Note on AI Use

Per the syllabus: AI-assisted debugging *after* you've attempted the problem yourself is fine. Having AI write these four methods for you is not — and it also won't prepare you for explaining this exact kind of code in your Midterm Project presentation in a few weeks. If you get stuck, the better move is office hours or the Moodle discussion forum.

Suggested Pacing

Not graded checkpoints — just a sane way to not do this all the night before.

By...	You should have...
Day 2-3	Run the starter file, confirmed matching output, read all four TODO specs, and understand what each is asking
Day 5-6	unique_ips() and multi_ip_users() done and tested
Day 8-9	busiest_hour() and generate_alerts() done and tested
Day 10-11 (due date)	README written, ZIP or GitHub link ready, submitted on Moodle

Grading (Suggested)

Component	Points
unique_ips() correct	4
multi_ip_users() correct	4
busiest_hour() correct	4
generate_alerts() — pulls the others together sensibly	4
Visible test/print output for each method	2
README (what / how / your-words explanation)	2
Total	**20**